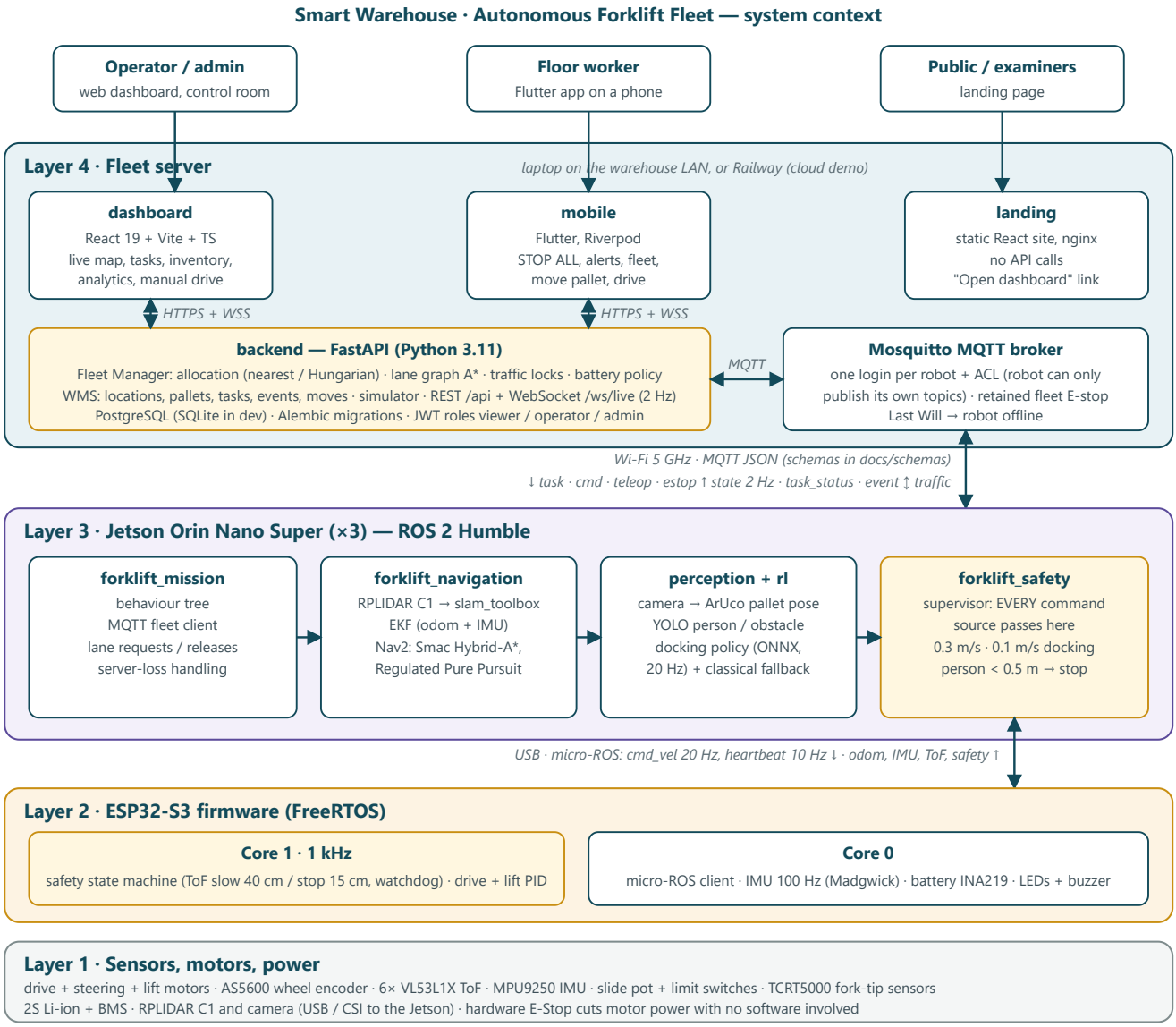
System architecture — Smart Warehouse & Autonomous Forklift Fleet

A 1:14-scale warehouse in which three autonomous forklifts (plus one spare) move pallets between racks, inbound and outbound docks without human help, under a central fleet server and a live control-room dashboard. Printable copy: this document.

Scale	1:14 · floor 6 × 4 m · forklift Huina 1577 Pro, 52.4 × 18.7 × 31.7 cm
Fleet	3 forklifts + 1 spare · 26 locations today (rack, dock, staging, charger) · 10–20 pallets
Per forklift	ESP32-S3 (real-time control and safety) + Jetson Orin Nano Super (ROS 2, perception, AI)
Server	FastAPI fleet manager + WMS, Mosquitto MQTT, PostgreSQL
Clients	React dashboard, Flutter phone app, public landing page
Code	7 public repositories in the Smart-Warhouse GitHub organisation

1. System context

Four layers. Each layer only talks to the one next to it, and every layer can stop the robot on its own.



Why the server does not use ROS. The server speaks plain MQTT + JSON, so it was built and demoed before any forklift existed. Its built-in simulator drives fake robots over the same lane graph, traffic locks and task lifecycle that real robots use. A real forklift joins by setting `simulated: false` in the map file.

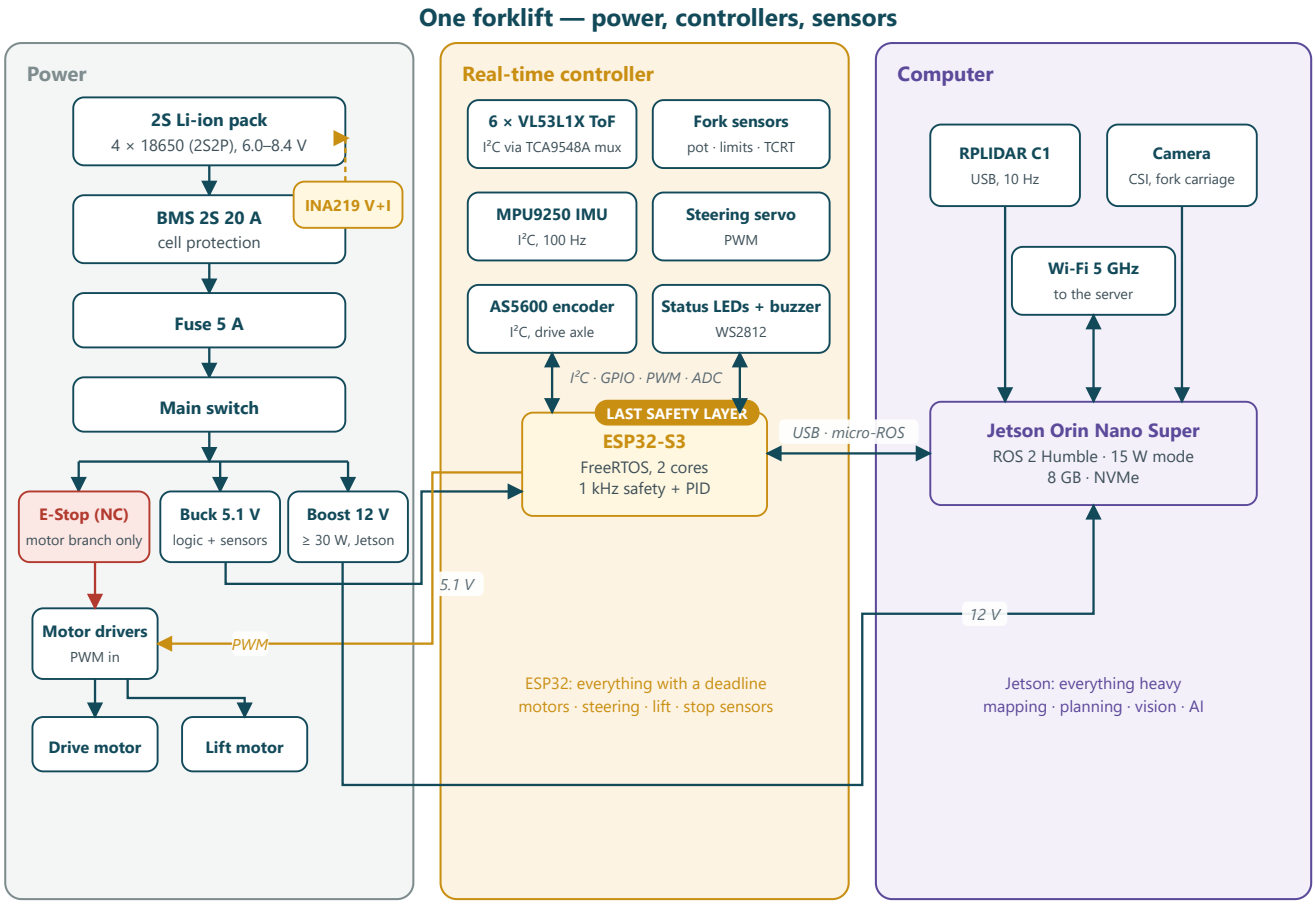
2. Repositories

Repository	Contents	Stack	State
backend	Fleet manager, WMS, REST + WebSocket API, simulator, MQTT bridge, Mosquitto config, integration + perf tests, allocation study	Python 3.11, FastAPI, SQLAlchemy, Alembic, aiomqtt	built, tested, deployed
dashboard	Control-room console: live map, fleet, tasks, inventory, analytics, events, manual drive, settings, kiosk	React 19, Vite, TypeScript, Tailwind	built, deployed
mobile	Floor app: STOP ALL, alerts, fleet, mini map, move pallet, manual drive	Flutter, Riverpod, go_router, Dio	built, APK
landing	Public project site, Arabic + English	React, Vite, nginx with strict CSP	built, deployed
robot	ESP32 firmware, ROS 2 workspace (10 packages), Gazebo twin, CAD for printed mounts	PlatformIO / FreeRTOS, ROS 2 Humble, Gazebo Fortress, CadQuery	written, host-tested, no hardware yet

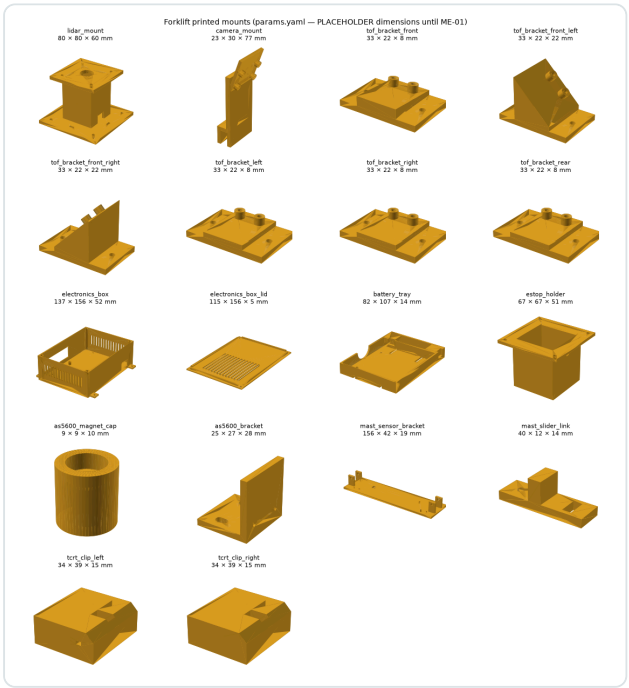
Repository	Contents	Stack	State
ai	Docking simulator and training, YOLO data/training pipeline	MuJoCo, Gymnasium, Stable-Baselines3, ONNX, Ultralytics	trained in simulation
docs	Requirements, contracts, JSON schemas, plan, thesis, guides, brand	Markdown	living

The repositories sit side by side in one workspace folder; tests that need a sibling (for example the robot tests that read the backend's map) find it there, and CI checks the sibling out the same way.

3. One forklift: hardware



The hardware E-Stop cuts the motors only: the ESP32 and Jetson stay powered and report why the forklift stopped



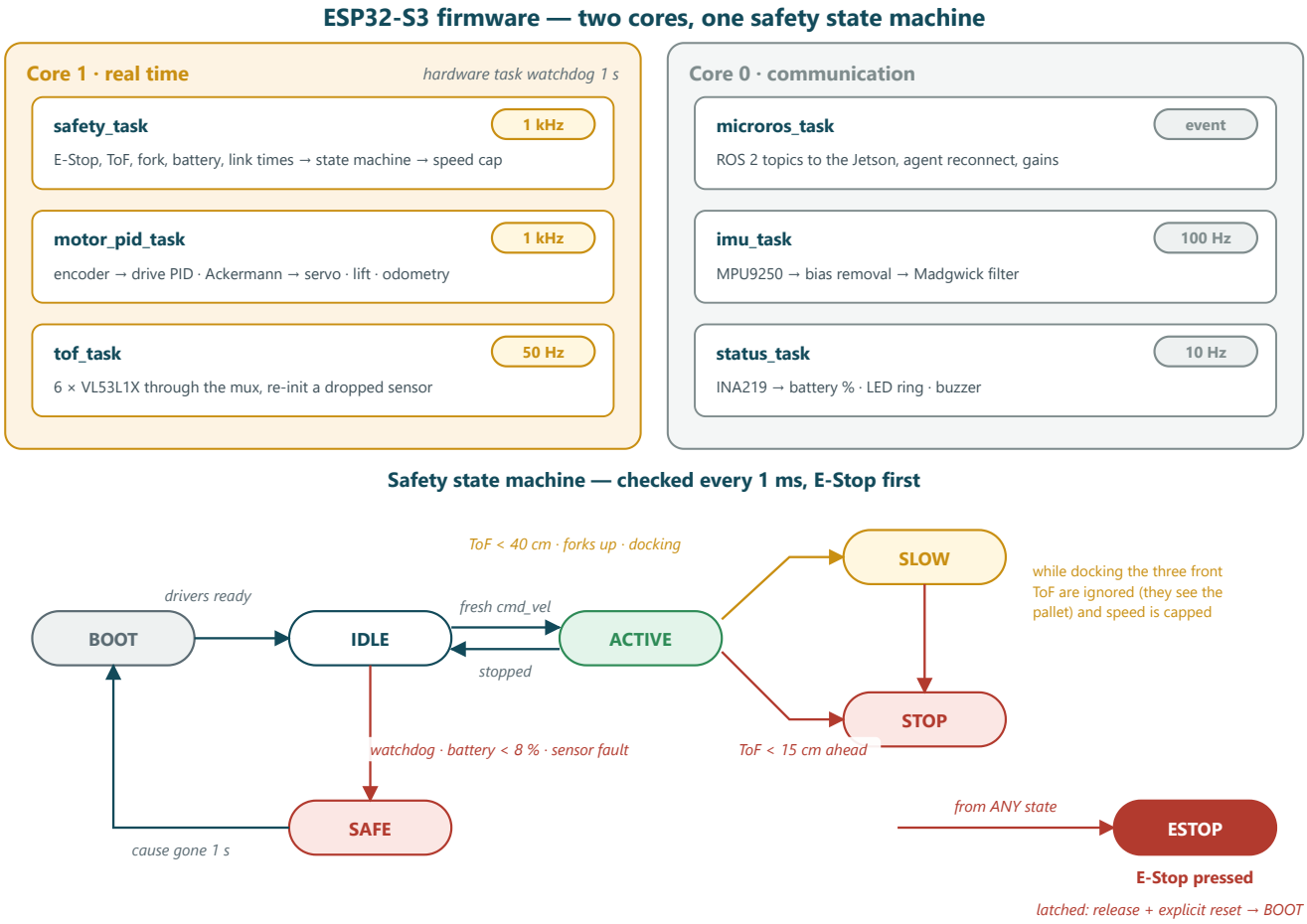
The 18 printed mounts, generated with CadQuery from `robot/cad/params.yaml` (placeholder sizes until the forklift is measured).

- **The E-Stop cuts only the motor branch.** The Jetson and ESP32 stay powered, so they can still log and report why the forklift stopped (EL-02).

- **The ESP32 owns everything with a deadline:** motors, steering, lift, and the short-range sensors that stop the robot.
- **The Jetson owns everything heavy:** LiDAR, camera, mapping, planning, AI.

4. Layer 2 — ESP32-S3 firmware

FreeRTOS on both cores. All decision logic (`forklift_core`) is plain C++ tested on the laptop (60 test cases); the board build runs in CI.



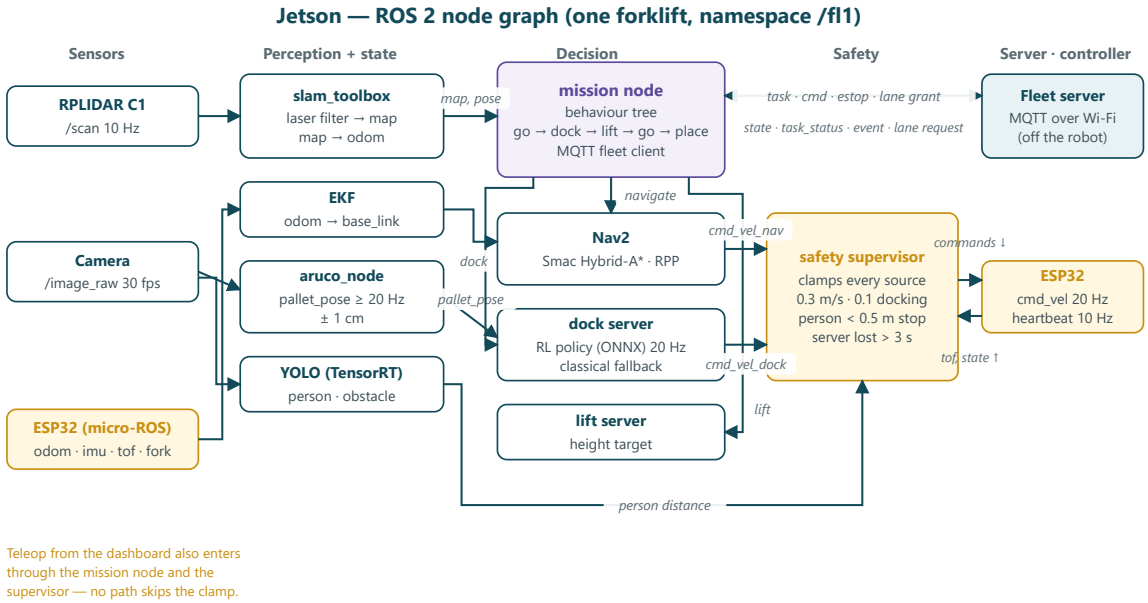
The E-Stop is latched: releasing the button alone does nothing. While docking, the three front ToF sensors are ignored, because they would see the pallet the forklift is about to pick up (SF-03). Speed is capped instead.

micro-ROS topic	Direction	Rate
`cmd_vel` , `lift_cmd`	Jetson → ESP32	20 Hz
`heartbeat`	Jetson → ESP32	10 Hz (missing → SAFE)
`wheel_odom` / `imu/data`	ESP32 → Jetson	50 Hz / 100 Hz
`tof_ring` , `fork_state`	ESP32 → Jetson	20 Hz
`safety_state`	ESP32 → Jetson	10 Hz + on change
`battery`	ESP32 → Jetson	1 Hz

5. Layer 3 — Jetson, ROS 2 Humble

Ten packages. Decision logic is plain Python without `rclpy`, so it is tested on the laptop; the ROS nodes are thin wrappers. The whole workspace builds and passes `colcon test` in a `ros:humble` container in CI.

Package	Role
`forklift_mission`	Behaviour tree for a task (go → dock → lift → go → place), MQTT fleet client, traffic requests, server-loss handling
`forklift_navigation`	slam_toolbox, EKF (wheel odometry + IMU), Nav2 with Smac Hybrid-A* (Reeds-Shepp) and Regulated Pure Pursuit, no rotate-in-place
`forklift_perception`	ArUco detection + solvePnP → `pallet_pose`; initial pose from location markers
`forklift_rl`	Docking policy (ONNX) with a classical fallback
`forklift_safety`	Supervisor that clamps every command source; bridge for fork state
`forklift_description`	URDF/Xacro, dimensions, TF tree `map → odom → base_link → {laser, camera, imu, fork}`
`forklift_bringup`	One launch file, YAML configs, namespaces `/fl1 /fl2 /fl3`, systemd autostart, rosbag recording
`forklift_interfaces`	Messages, services and actions (`Lift`, `Dock`, `MoveTask`, `Reset`, …)
`forklift_camera_stream`	Low-resolution MJPEG of the fork camera for manual driving
`forklift_sim`	Gazebo Fortress digital twin generated from the warehouse map

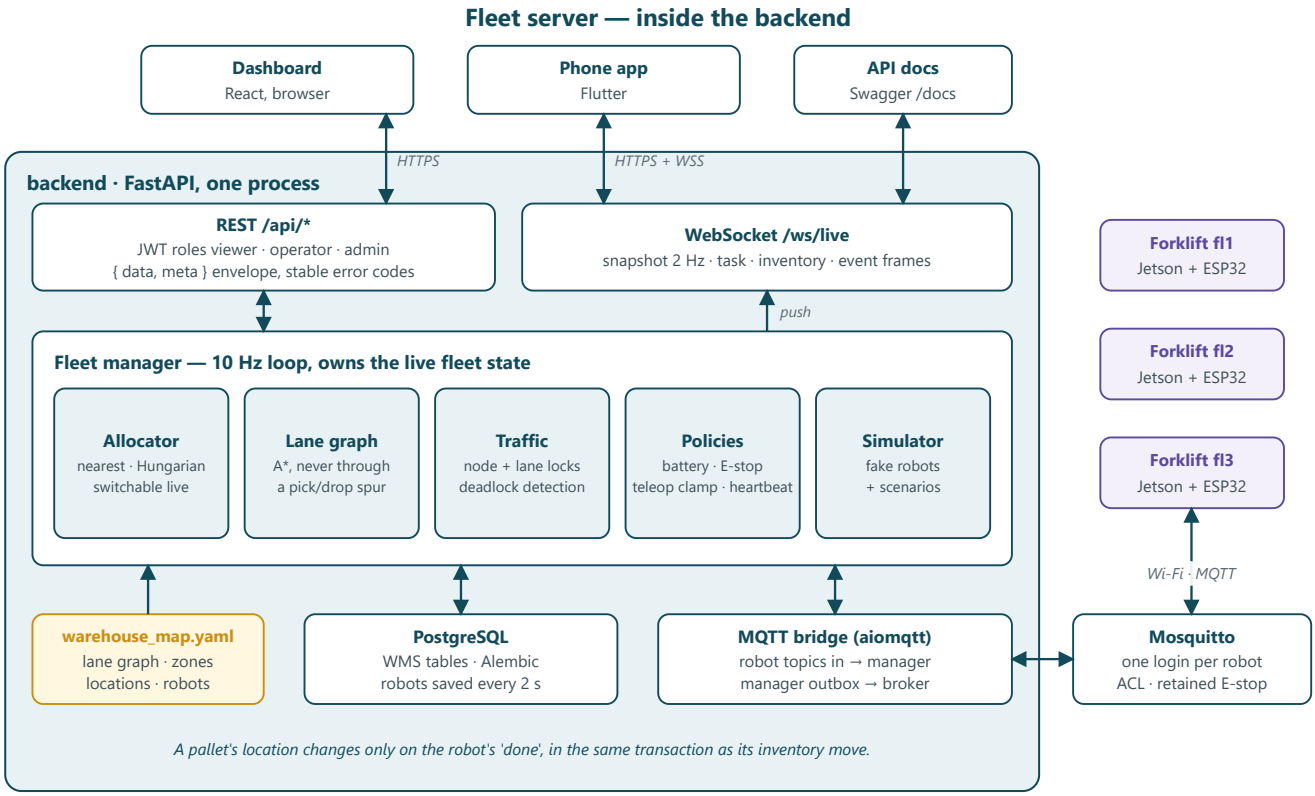


Teleop from the dashboard also enters through the mission node and the supervisor — no path skips the clamp.

Supervisor limits:

- **Speed:** 0.3 m/s normally, 0.1 m/s with the forks above 5 cm or while docking.
- **ToF ahead:** slow below 40 cm, stop below 15 cm, in the direction of travel.
- **Person (YOLO):** slow below 1 m, stop below 0.5 m.
- **Server lost for more than 3 s:** finish the current step, then stop.

6. Layer 4 — fleet server



Module	Responsibility
`fleet/graph.py`	Lane graph from the map file; A* that never routes through a pick/drop spur
`fleet/traffic.py`	One robot per lane and node, all-or-nothing lock acquisition, wait-for graph to detect deadlocks
`fleet/allocator.py`	`nearest` and optimal `hungarian` task assignment, switchable at run time
`fleet/manager.py`	Robot state, task lifecycle, commands, fleet E-stop, battery policy, teleop clamping, simulator
`fleet/mqtt_bridge.py`	Robot topics in → manager; manager outbox → broker
`routers/*`	REST endpoints and the WebSocket
`migrate.py`	Alembic upgrade on start-up

Data model (WMS)

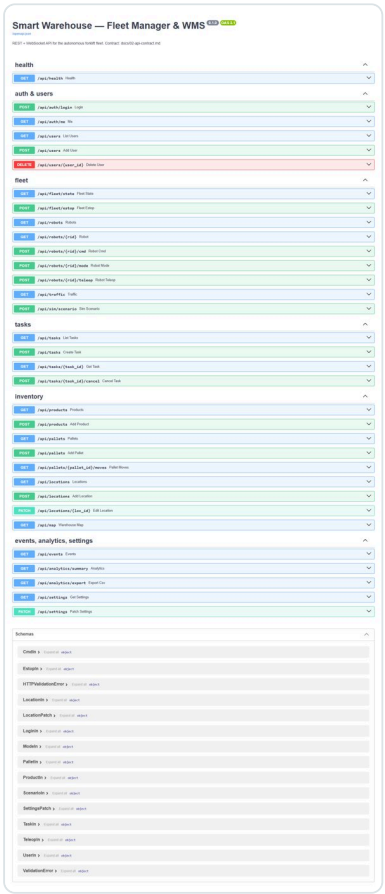
Table	Holds
`locations`	Code (`R1-W1-L2`), type, x/y/yaw, level, height, ArUco id, lane-graph node, occupied
`pallets / products`	Pallet (ArUco id 1–20, SKU, quantity, weight, location, status) and product catalogue
`robots`	Status, battery, pose, current task (live state in memory, saved every 2 s)
`tasks`	`T-0042` , pallet, from/to, robot, priority 1–5, status, route, distance, wait, timestamps, failure reason
`inventory_moves`	Every pallet move, written **in the same transaction** as the task's `done`
`events`	Safety / error / info events with severity, code and position
`traffic_waits`	Time spent waiting per lane, for the congestion heat map

Table	Holds
`users` / `settings`	Accounts with roles (viewer / operator / admin), runtime settings

REST API (summary)

Area	Endpoints
Auth	`POST /api/auth/login`, `GET /api/auth/me`
Fleet	`GET /api/fleet/state`, `POST /api/fleet/estop`, `GET /api/robots`, `POST /api/robots/{id}/cmd · /mode · /teleop`, `GET /api/robots/{id}/camera`
Tasks	`GET/POST /api/tasks`, `GET /api/tasks/{id}`, `POST /api/tasks/{id}/cancel`
Inventory	`GET/POST /api/pallets`, `GET /api/pallets/{id}/moves`, `GET/POST /api/products`, `GET/POST/PATCH /api/locations`
Map, traffic	`GET /api/map`, `GET /api/traffic`
Monitoring	`GET /api/events`, `GET /api/analytics/summary`, `GET /api/analytics/export` (CSV)
Admin	`GET/PATCH /api/settings`, `GET/POST/DELETE /api/users`, `POST /api/sim/scenario`
Health	`GET /api/health` (no login)

Every response uses one envelope, `{ "data": …, "meta": … }`, and errors carry a stable code.

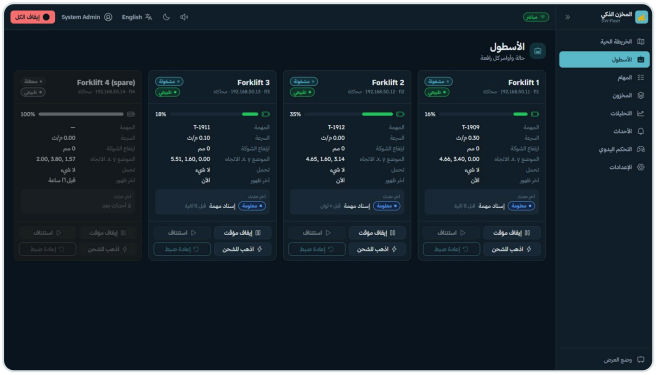


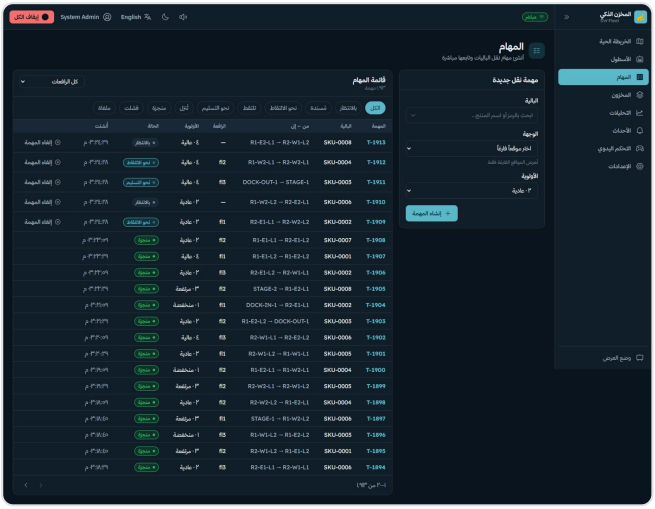
The backend serves interactive API docs (Swagger) for every endpoint.

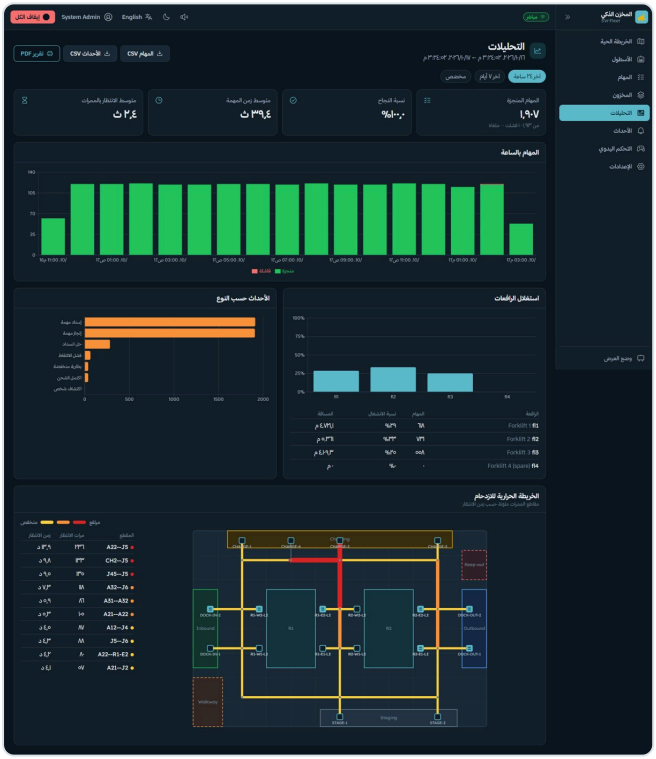
7. User interfaces

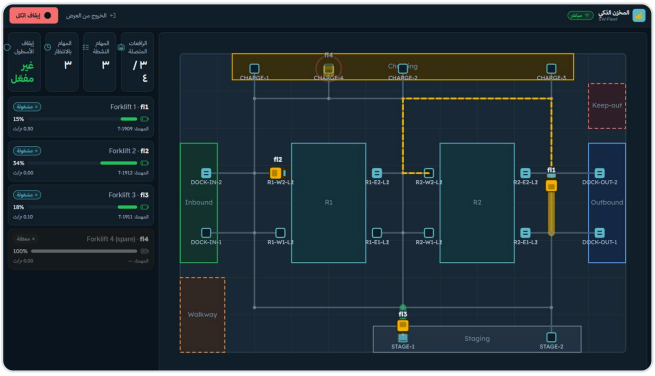
All three clients talk only to the fleet server: REST for actions, and the 2 Hz WebSocket for live state. Arabic is the default (right-to-left), English is one tap away, and light and dark themes are supported.

Dashboard (control room)









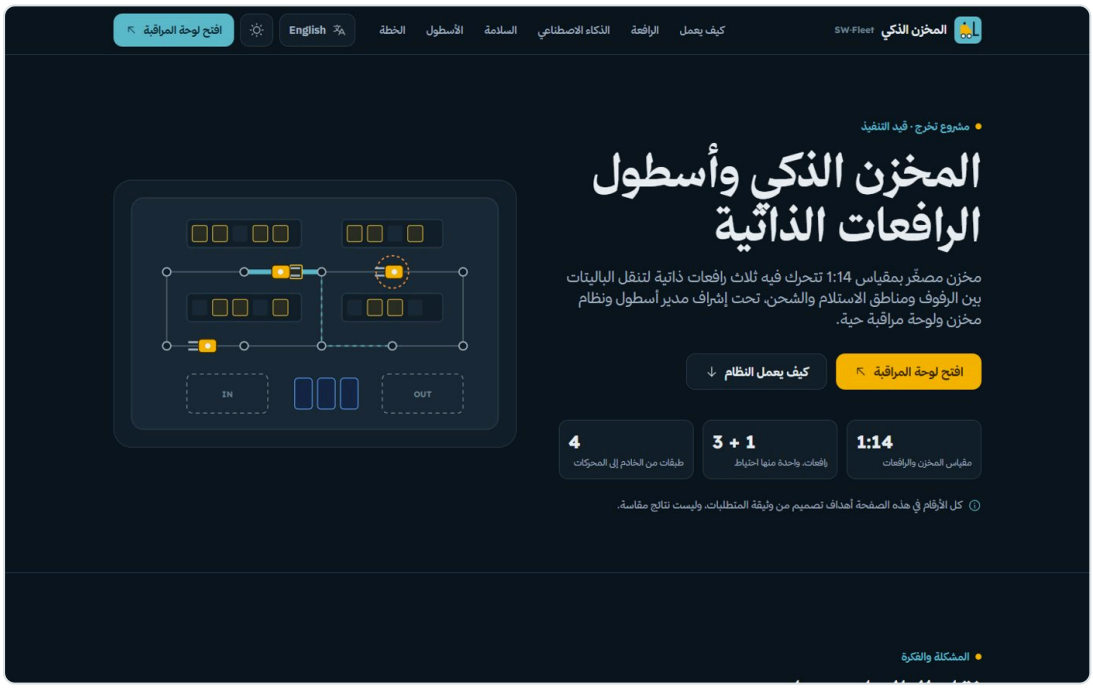
Fleet cards with commands, the task list, analytics with the congestion heat map, and the kiosk mode for the defence screen. The live map is shown in section 9.

Phone app (floor worker)



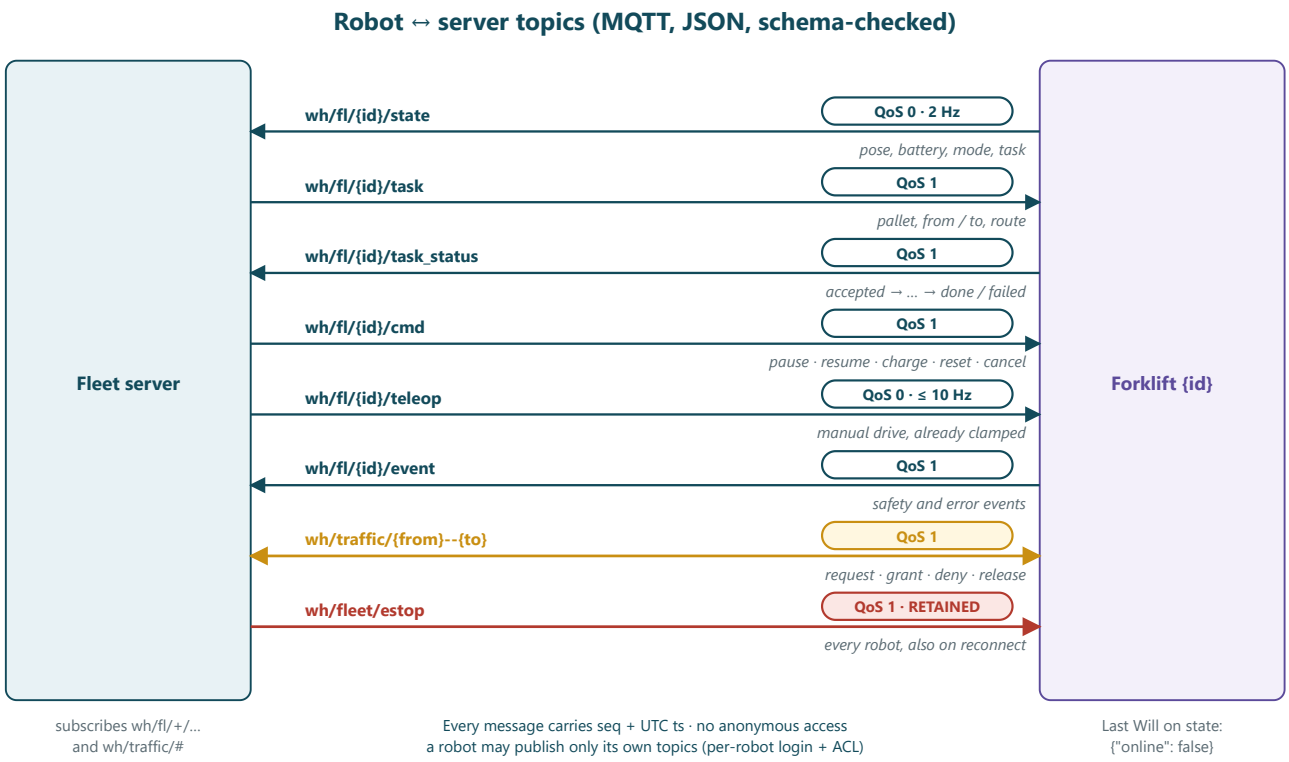
*Sign-in with the server address, fleet, mini map, tasks and safety alerts. **STOP ALL** is on every screen and needs a 1-second hold.*

Landing page (public)



Public project site: Arabic and English, static, served by nginx with a strict content-security policy.

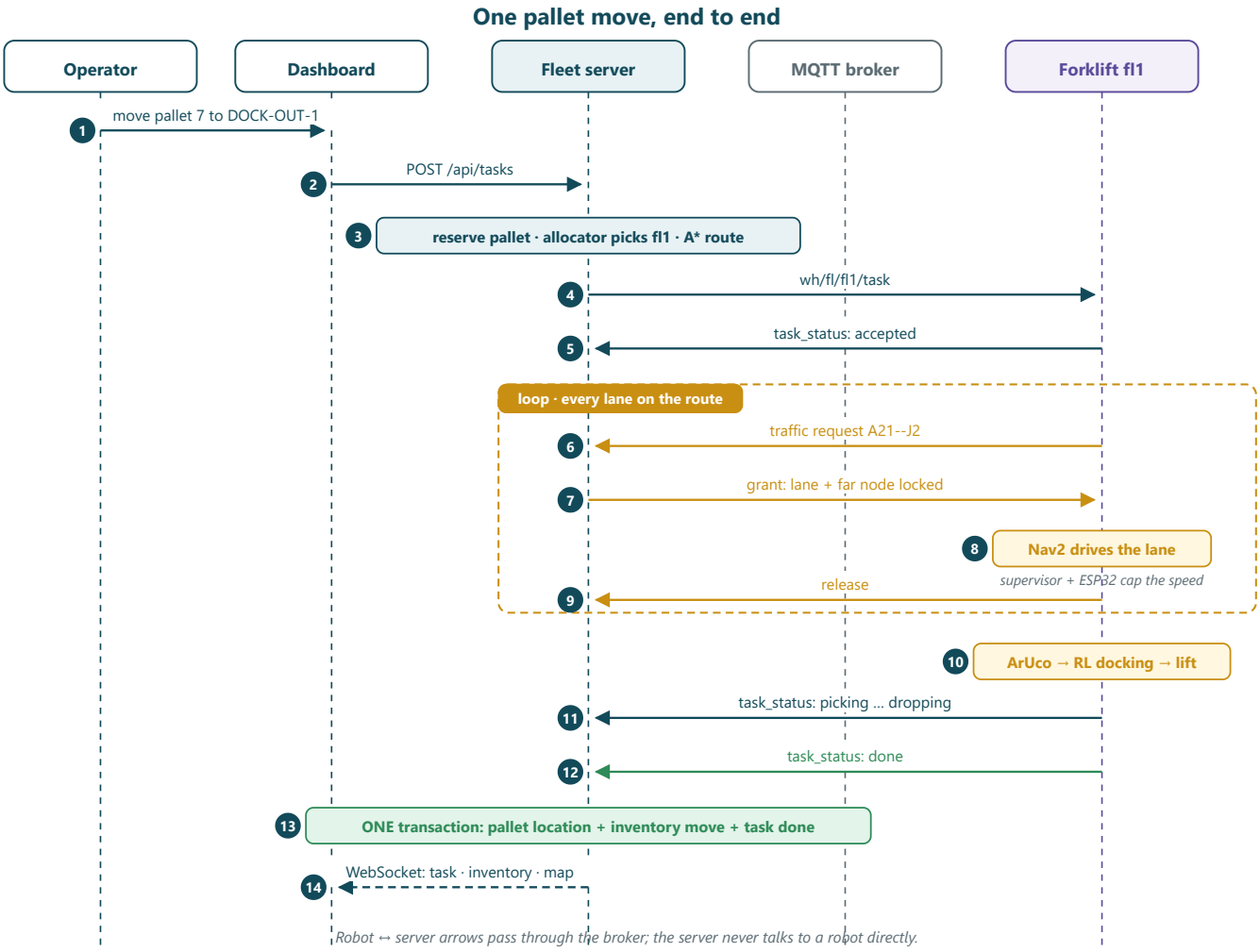
8. Robot ↔ server communication (MQTT)

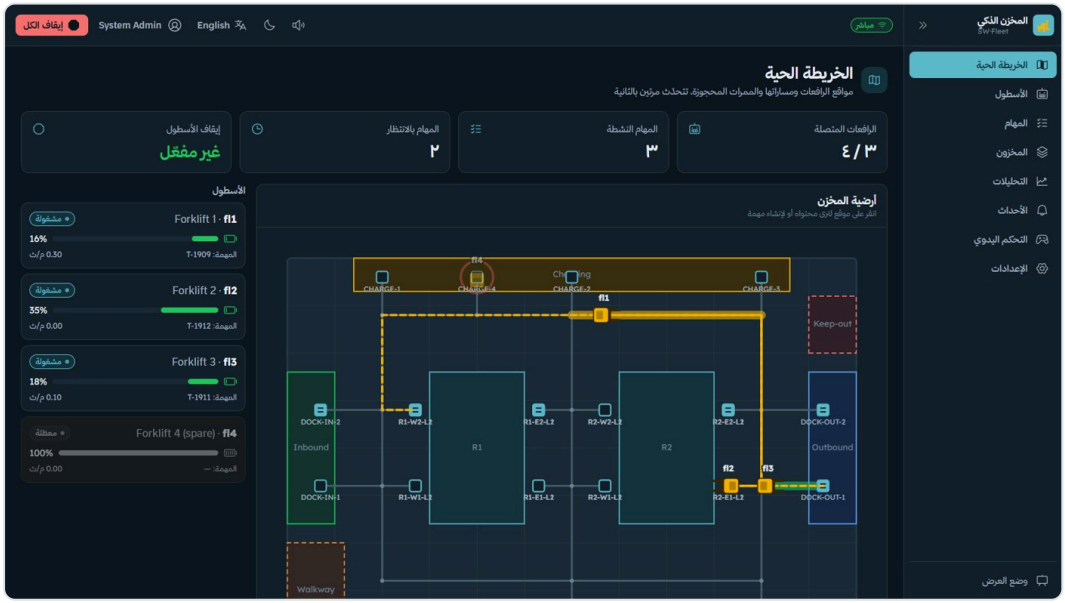


Topic	Direction	QoS	Notes
`wh/fl/{id}/state`	robot → server	0 · 2 Hz	Pose, battery, mode, task; Last Will `{"online": false}`
`wh/fl/{id}/task`	server → robot	1	Pallet, from/to with coordinates and height, route
`wh/fl/{id}/task_status`	robot → server	1	accepted → to_pickup → picking → to_drop → dropping → done / failed
`wh/fl/{id}/cmd`	server → robot	1	pause, resume, go_charge, reset, cancel
`wh/fl/{id}/teleop`	server → robot	0 · ≤ 10 Hz	Manual drive, already clamped by the server
`wh/fl/{id}/event`	robot → server	1	Safety and error events
`wh/traffic/{from}--{to}`	both	1	request / grant / deny / release of a lane
`wh/fleet/estop`	server → all	1 · **retained**	A robot that reconnects learns the E-stop state at once

Every message carries a sequence number and a UTC timestamp, and is validated against a JSON schema in docs/schemas. The broker has no anonymous access. Each robot has its own login, and the ACL lets a robot publish only its own topics.

9. End to end: one pallet move

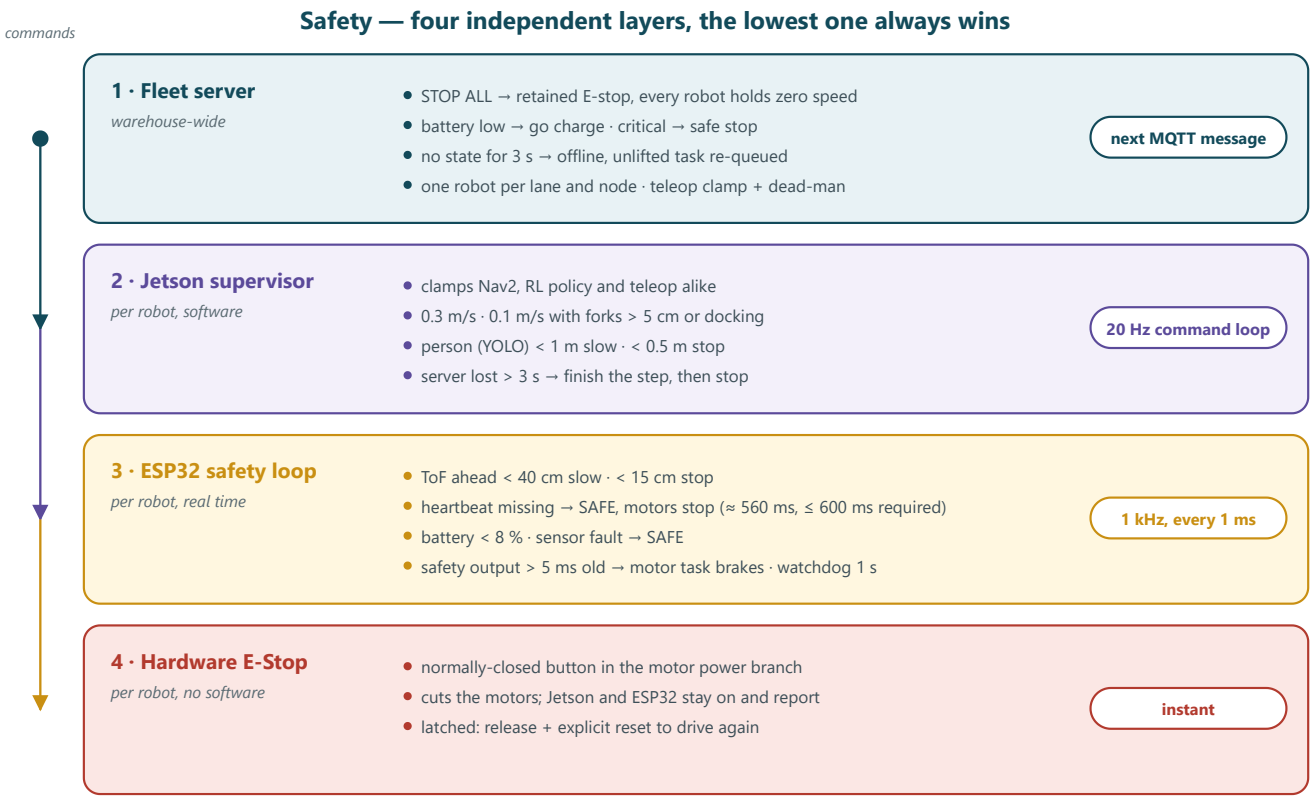




The same flow on the live map: each forklift's planned path and reserved lanes are highlighted.

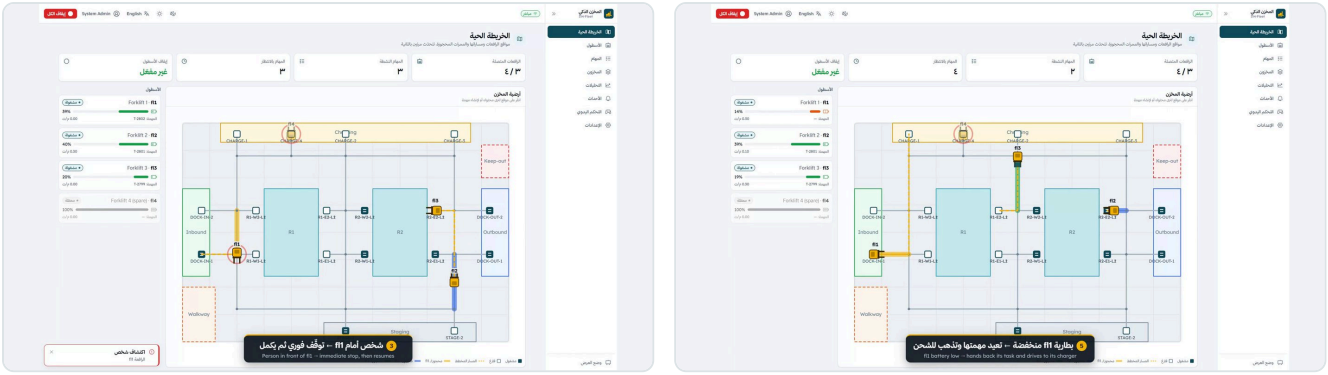
10. Safety architecture

Defence in depth: each layer assumes the one above it can fail.

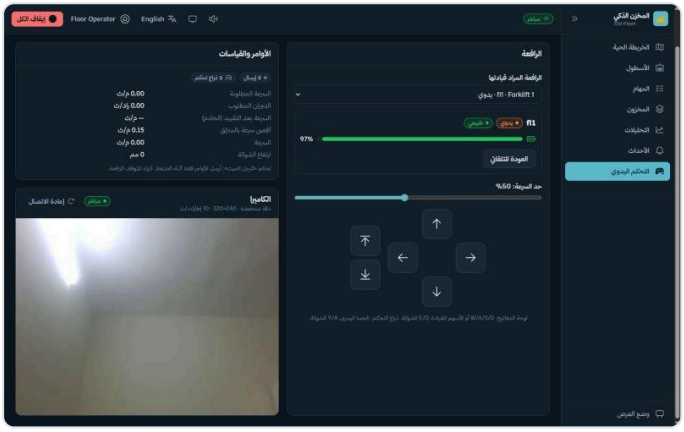


Every command, from the dashboard, Nav2, the RL policy or the gamepad, passes every layer below it. A lower layer never trusts the one above.

Failure	What happens
Person in front	YOLO → supervisor slows < 1 m, stops < 0.5 m; ToF stops < 15 cm anyway
Wi-Fi or server lost	Robot finishes the current lane, stops, waits, and resumes when the link returns
Robot disconnects	Broker sends its Last Will → server marks it offline; an unlifted task goes to another robot
Jetson crashes or USB unplugged	ESP32 misses the heartbeat → SAFE, motors stop (≈ 560 ms on the model, ≤ 600 ms required)
ESP32 control task stalls	Motor task brakes if the safety output is > 5 ms old; hardware watchdog reboots after 1 s
Two robots want the same lane	Only one gets the lock; deadlocks are detected and broken by detour or back-off
Battery low / critical	Low → go charge; critical (< 8 %) → safe stop
Operator presses STOP ALL	Retained E-stop: every robot holds zero speed until released

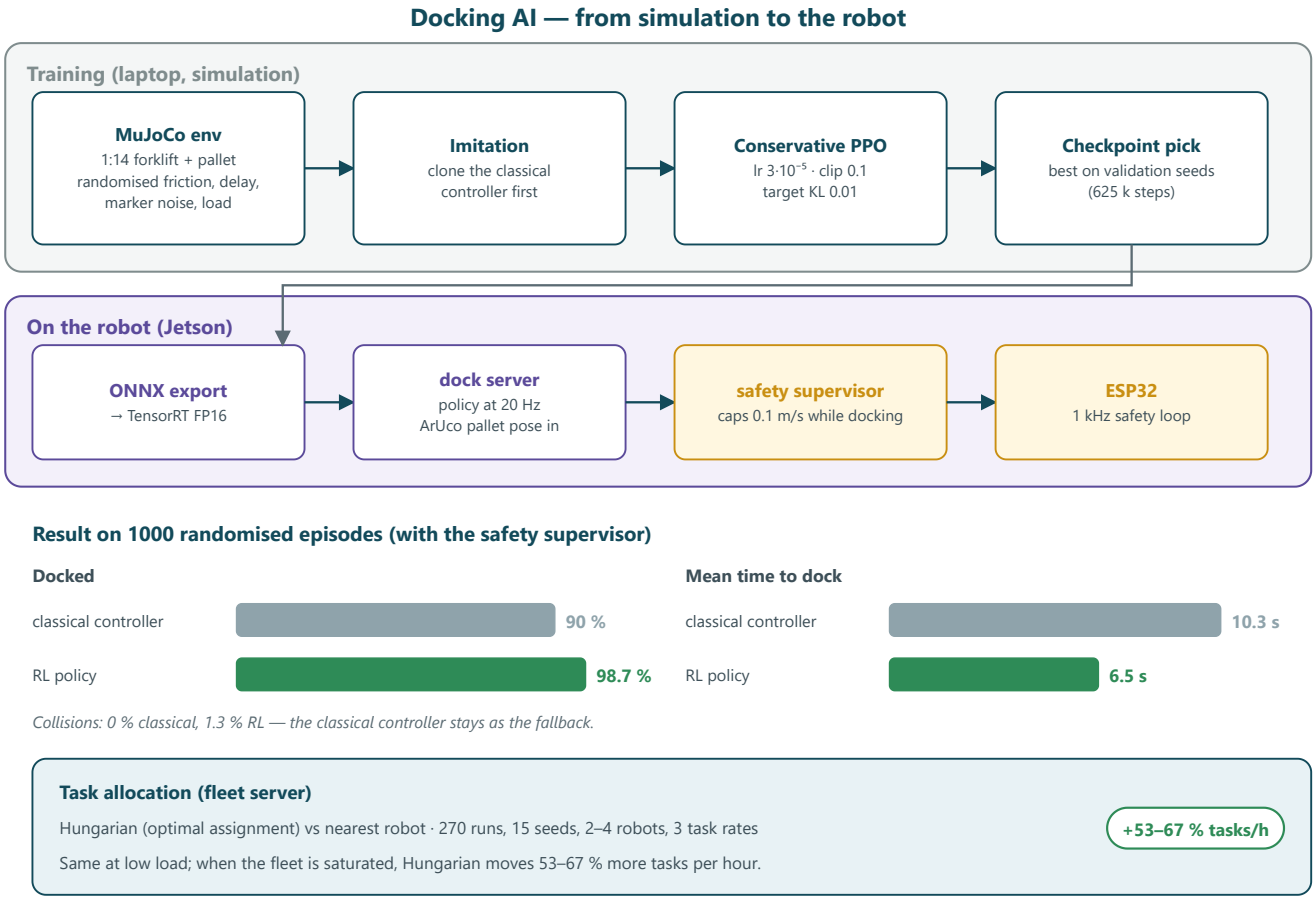


Simulator scenarios on the dashboard. Left: a person in front of fl1, so it stops at once and then resumes. Right: fl1's battery is low, so it hands back its task and drives to its charger.



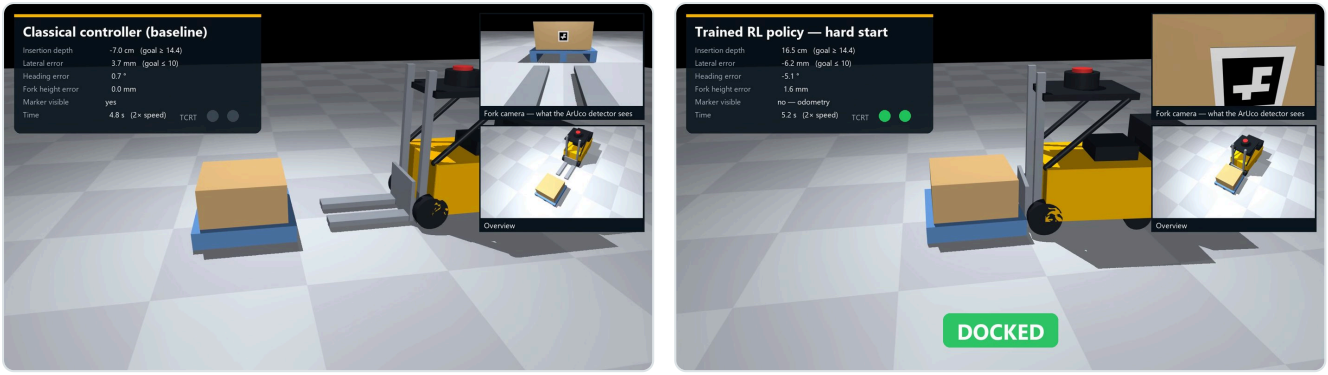
Manual drive: the fork camera is streamed only while the forklift is in manual mode, and every command is clamped by the server and the robot.

11. AI components

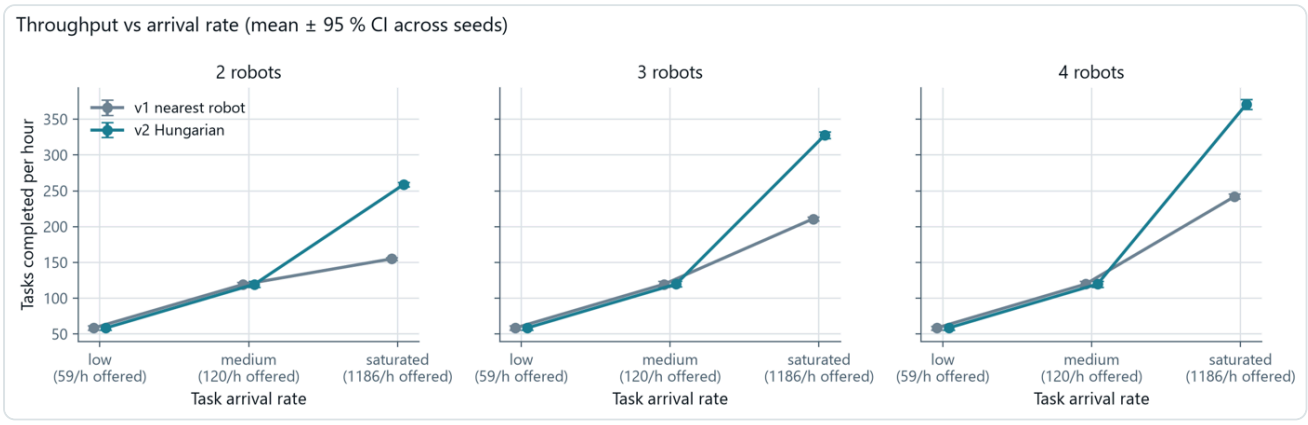


Component	Method	Result (simulation)
Pallet docking	MuJoCo environment with domain randomisation; imitation of the classical controller, then conservative PPO; checkpoint chosen on validation seeds; exported to ONNX	**98.7 %** docked, 6.5 s (with the safety supervisor), vs 90 %, 10.3 s for the classical controller, on 1000 episodes
Task allocation	Hungarian (optimal assignment) vs nearest robot	Same at low load; when saturated, **53–67 % more tasks/h** (270 runs, 15 seeds)
People / obstacles	YOLO11n, TensorRT FP16 on the Jetson	Starter model (COCO persons + synthetic images): mAP@0.5 0.848, but person recall 0.68; ≥ 400 real images per class still needed
Pallet pose	ArUco + solvePnP, sub-pixel corners	≤ 1 cm, ≤ 2° required at ≤ 50 cm

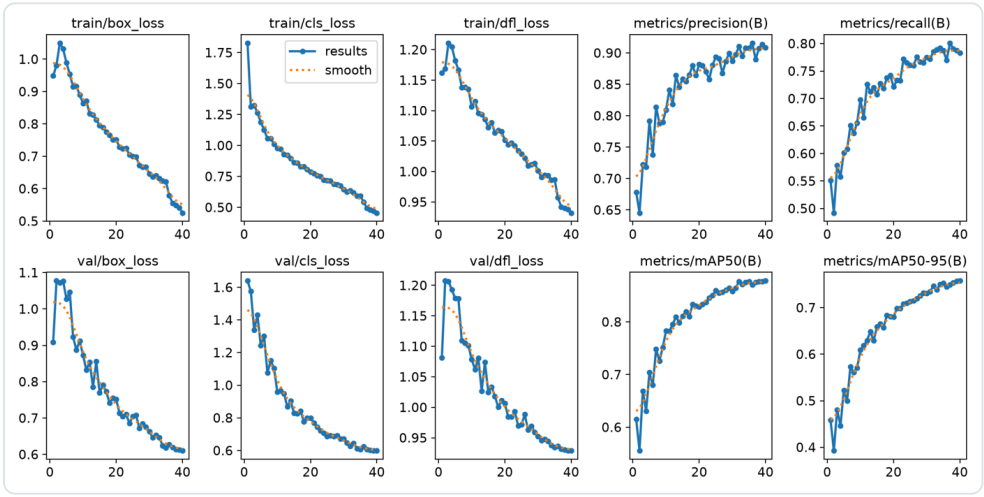
The AI never drives the motors directly: every command it produces goes through the safety supervisor and the ESP32 safety loop, like any other command.



The MuJoCo docking simulator. Left: the classical controller. Right: the trained RL policy docked from a hard start. The small views show what the fork camera's ArUco detector sees.



Allocation study: tasks per hour against the task rate, Hungarian vs nearest robot.



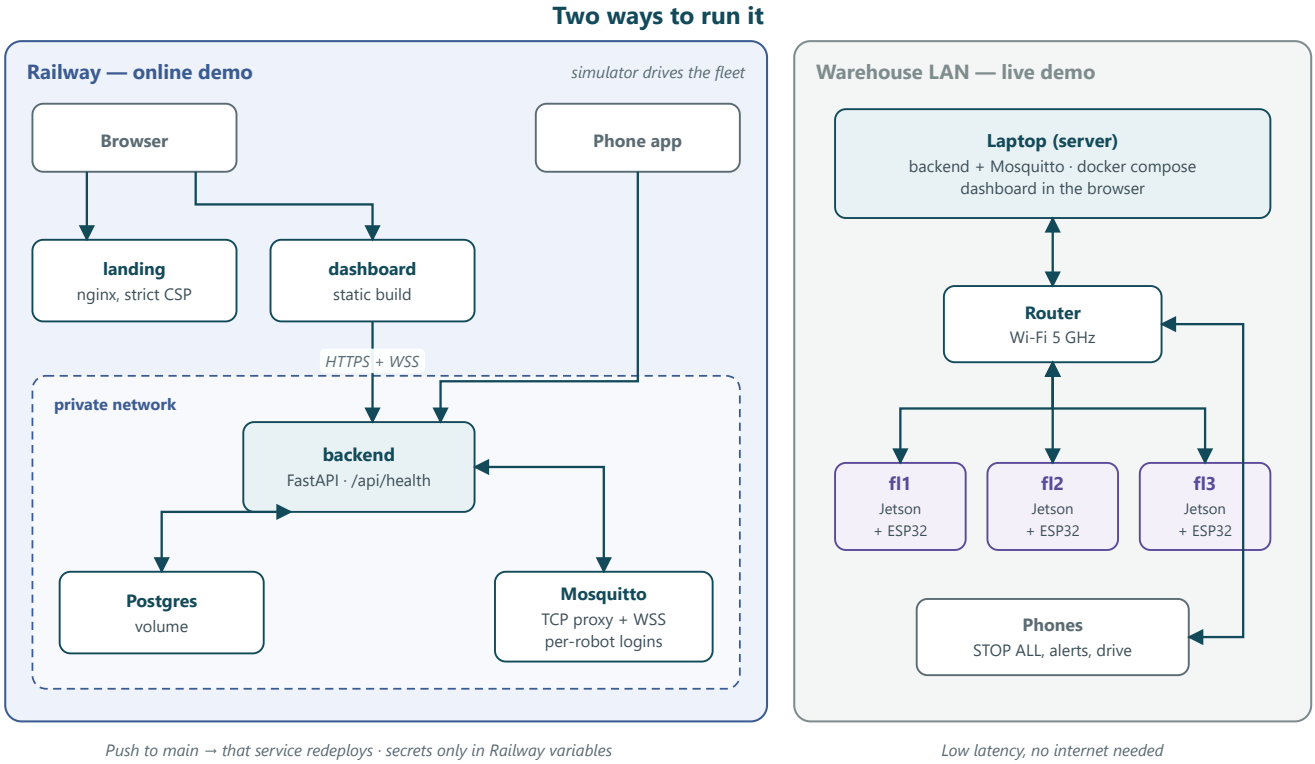
Training curves of the starter YOLO11n model (40 epochs). This is not the final acceptance result.

12. Simulation and testing

Level	Tool	What it proves
Fleet logic	Built-in server simulator	Allocation, traffic, deadlocks, battery, failures — no hardware
Robot ↔ server	Real Mosquitto + reference robot client	Logins, ACL, Last Will, retained E-stop, broker restart, 3 robots × 20 tasks
Docking	MuJoCo + Gymnasium	Training and evaluation of the docking policy
Whole robot	Gazebo Fortress twin of the warehouse	Nav2 + mission + server against simulated forklifts
Firmware	Host unit tests + board build	Safety state machine, kinematics, PID, IMU filter

CI runs on every push in each repository (13 jobs across the organisation): unit tests, type checks, lint, builds, the ROS 2 `colcon` build and tests in `ros:humble`, and the integration suite against a real Mosquitto container.

13. Deployment



- **Railway (cloud demo):** the online demo, with the simulator driving the fleet. Every push to `main` redeploys the matching service. Secrets live only in Railway's service variables.
- **Warehouse LAN (live demo with real forklifts):** the same backend and broker run on a laptop on the warehouse Wi-Fi. That gives lower latency and no dependence on the internet.

14. Key design decisions

Decision	Choice	Why
Robot ↔ server protocol	MQTT + JSON schemas, not ROS across the network	Server built and tested without robots; robust over Wi-Fi; Last Will and retained messages give safety for free
Traffic control	Lane graph with node + lane locks	Predictable, testable, no gridlock; robots never block an aisle while docking (spurs)
One source of truth for the floor	`warehouse_map.yaml`	Same file drives the server, dashboard, Gazebo world and occupancy map
WMS consistency	Pallet moves only on `done`, in one transaction	Inventory always matches reality, even if a task fails half-way
Where AI is used	Docking, allocation, vision only	Navigation, control and safety stay classical and verifiable
Safety	Four independent layers, hardware E-Stop last	No single software failure can keep the motors running
Adding a robot	One entry in the map file	No code change (NFR-09)